FINAL PROJECT PRESENTATION · GROUP 2

RentEZ

Car rental on five independently deployable microservices

John Ling · Sachin Agrawal · Quek Shen An · Muhammad Nur Hakim Bin Azman

Graduate Certificate in Architecting Scalable Systems · 16 Oct 2026

github.com/Uygnis/NUS-ISS-ASS-Practice-Module

AGENDA

Agenda

#	Section	What we cover
01	Overview	Business problem, platform thinking, scope
02	Project Conduct	Status, outstanding issues, milestones, team
03	Logical Architecture & Design	Key decisions, bounded contexts, DDD
04	Physical Architecture & Design	AWS deployment, persistence, book-and-pay
05	DevOps & Development Lifecycle	Source control, pipeline, findings
06	Demo	Use cases, performance, autoscaling
	Q&A	Questions and answers

01 OVERVIEW · 2 + 1 MIN

Context and Business Problem

Context

- Customers browse, book, pay, get notified
- Staff run the fleet; admins run users and refunds
- Five services that cannot read each other's tables

Business problem

- A car exists once: no double-booking
- Money and bookings must always agree
- A notification outage must not fail a booking

Our solution: every booking is guaranteed, every payment matches it, and capacity grows with demand

01 OVERVIEW · 2 + 1 MIN

Platform Thinking

Producers

- Staff publish cars, maintenance
- Services publish events

Platform

- Date-range availability
- Booking + payment saga

Consumers

- Customers book and pay
- Notification reads events

Network effect: more cars draw more customers

Shared services: five services, each deployable alone

Open interfaces: REST APIs + outbox domain events

01 OVERVIEW · 2 + 1 MIN

Project Scope

In scope

- Customer, staff and admin flows
- AWS deployment: dev + prod
- CI/CD with SAST + SCA
- k6 load and scalability tests

Out of scope

- Real payment provider (in-process mock gateway)
- Refresh tokens and token revocation
- Queue-based autoscaling (KEDA), multi-region

Assumptions: student AWS budget (spot nodes, daily teardown); depth on correctness and scalability over feature breadth

Current Status and Outstanding Issues

OVERALL COMPLETION

100%

5 of 5 services + frontend deployed

Outstanding issues

Issue	Impact	Plan / owner
CPU-based HPA lags pool exhaustion	High	KEDA on pending connections / future
Frontend tests off in CI	Medium	Re-enable npm test
No token revocation or refresh	Medium	DynamoDB sessions / defer
CloudFront masks API 404s as 200	Low	Scoped /api/* behaviour / defer

02 PROJECT CONDUCT · 1.5 + 0.5 MIN

Milestones and Team Effort

M1 · Services
Aug · Done

M2 · Frontend
29 Aug · Done

M3 · AWS + CI/CD
Mid Sep · Done

M4 · Load testing
27 Sep · Done

M5 · Final demo
16 Oct · Planned

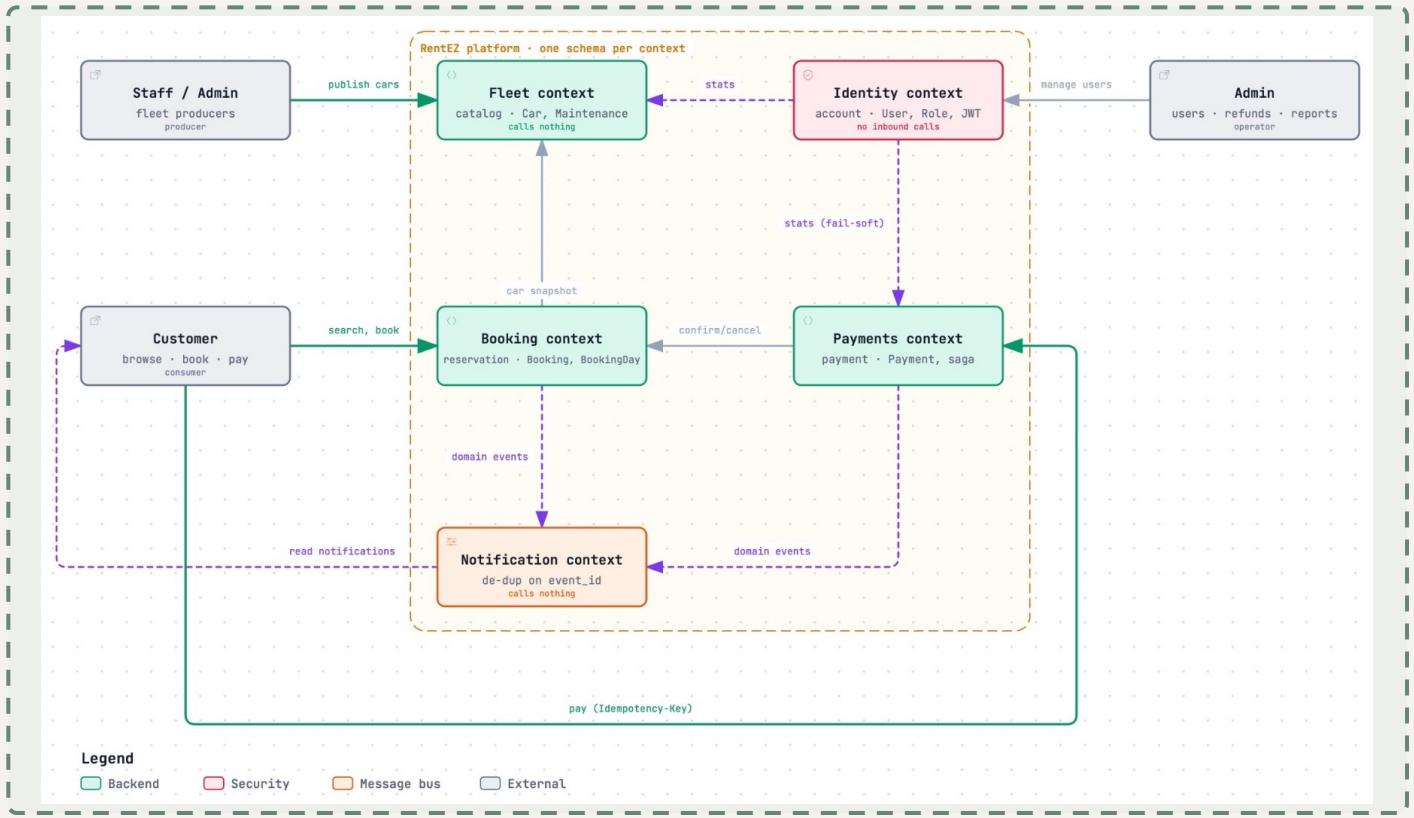
Member	Role	Main contributions	Effort (h)
John Ling	DevOps	AWS infrastructure, CI/CD, SonarCloud	[__]
Sachin Agrawal	Frontend	React SPA, booking and admin pages	[__]
Quek Shen An	Architecture	Service boundaries, saga, load testing	[__]
Muhammad Nur Hakim Bin Azman	Backend	Spring Boot services and APIs	[__]
Total			[__]

Key Architectural Decisions: Logical

ID	Decision	Rationale	Trade-off
AD-01	Database per service (schema + role, no grants)	A boundary you can cross is not a boundary; db-isolation proves it	One RDS instance shared; no joins
AD-02	No cross-service joins: snapshot or call the owner	Schemas stay private; bookings survive repricing	Duplicated car fields on booking
AD-03	Saga + idempotency, no distributed transactions	2PC blocks and does not work over HTTP	Compensation + reconciliation code
AD-04	No dependency cycles: availability lives in reservation	Every service builds, tests and deploys alone	One endpoint moved out of catalog
AD-05	Stateless JWT validated locally	Identity is never a bottleneck or SPOF	Permissions only as fresh as the token (15 min)
AD-06	Degrade, don't cascade (partial: true, null not 0)	A confident zero is worse than "unknown"	Clients must handle missing sections

NUS National University of Singapore | ISS

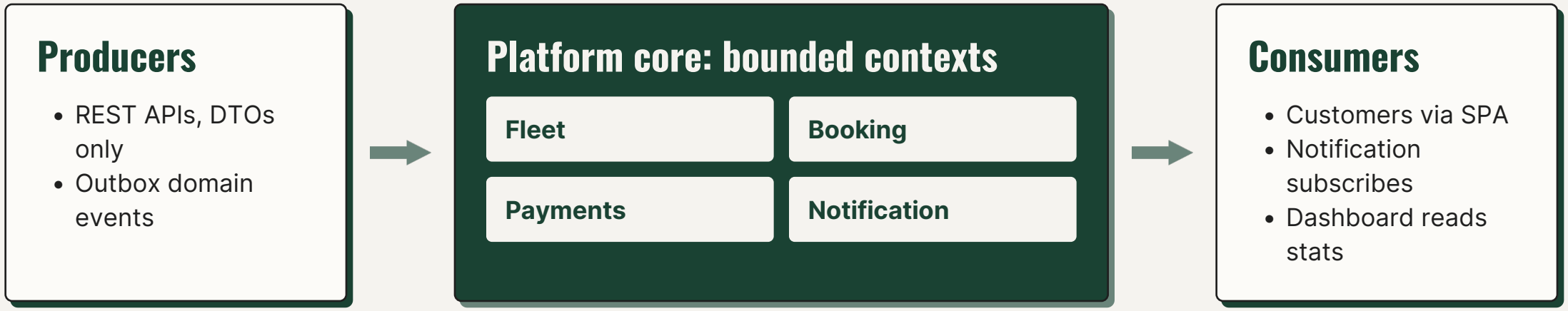
Logical Components



Key components

- **Fleet** cars, maintenance; calls nothing
- **Booking** availability, bookings, snapshot
- **Payments** payment saga, refunds
- **Notification** consumes events; calls nothing
- **Identity** JWT, admin dashboard; no callers

Domain-Driven Design for the Platform



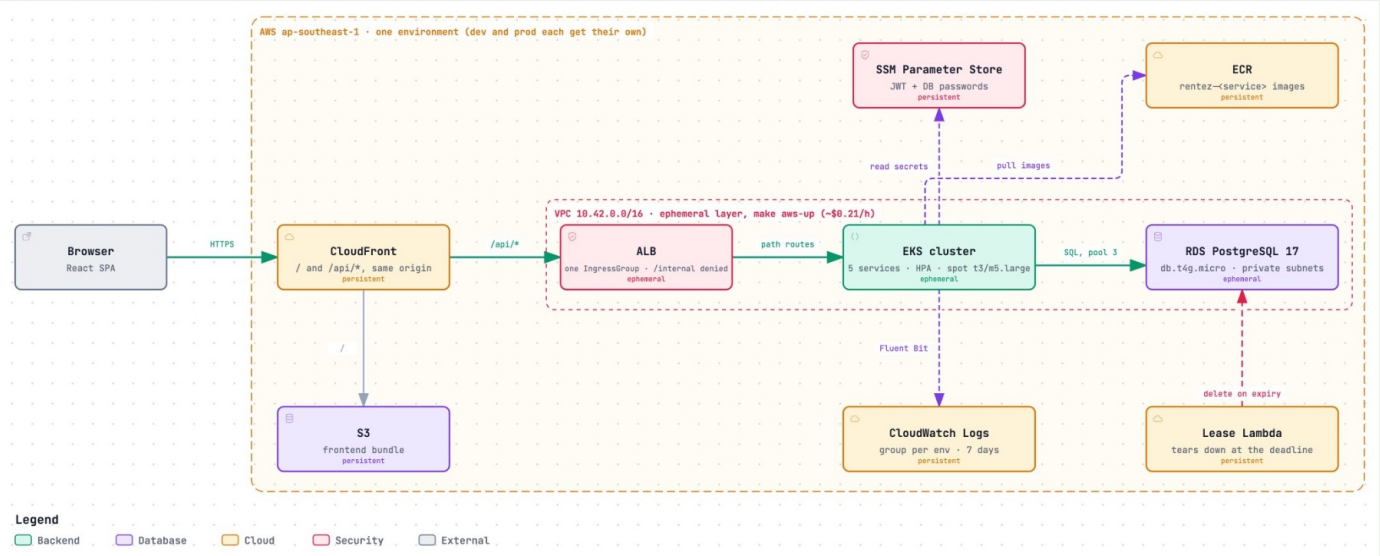
DDD PATTERNS USED

- Aggregates: Booking
- Events: outbox
- ACL: car snapshot
- Open host: Fleet API · JWT claims

Key Architectural Decisions: Physical

ID	Decision	Rationale	Trade-off
PD-01	CloudFront as the only public entry (/ and /api/*)	Same origin: no CORS, certificate or domain	Edge rewrites 404s for the SPA
PD-02	One ALB via IngressGroup; spot nodes; no NAT	~70% cheaper; pods are stateless	Spot interruptions; public subnets + SGs
PD-03	Persistent vs ephemeral layers + lease Lambda	$0.80/month idle, $0.21/hour running; no runaway bill	~20 min to bring the cluster up
PD-04	Separate dev and prod (own cluster, RDS, logs)	A load test on dev cannot starve the prod demo	Two of everything ephemeral
PD-05	HPA per service + Cluster Autoscaler (2-5 nodes)	Capacity follows demand automatically	Notification needs queue-based scaling

Physical Architecture



Network

VPC · one ALB · private RDS

Nodes

EKS · 2-5 spot nodes

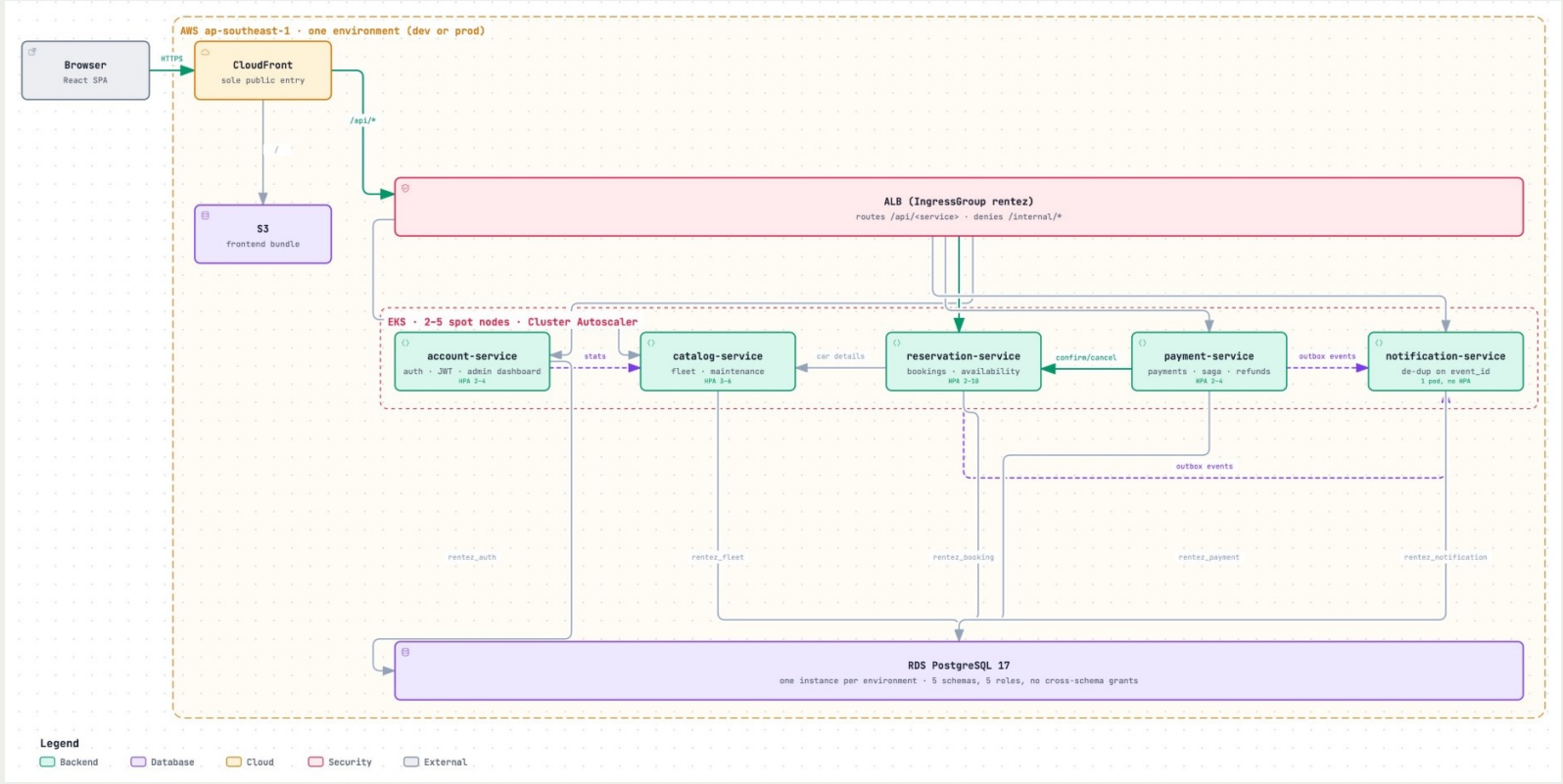
Technologies

Java 21 · Spring Boot · React

Cloud services

CloudFront · S3 · ECR · RDS

System Overview: Services on AWS



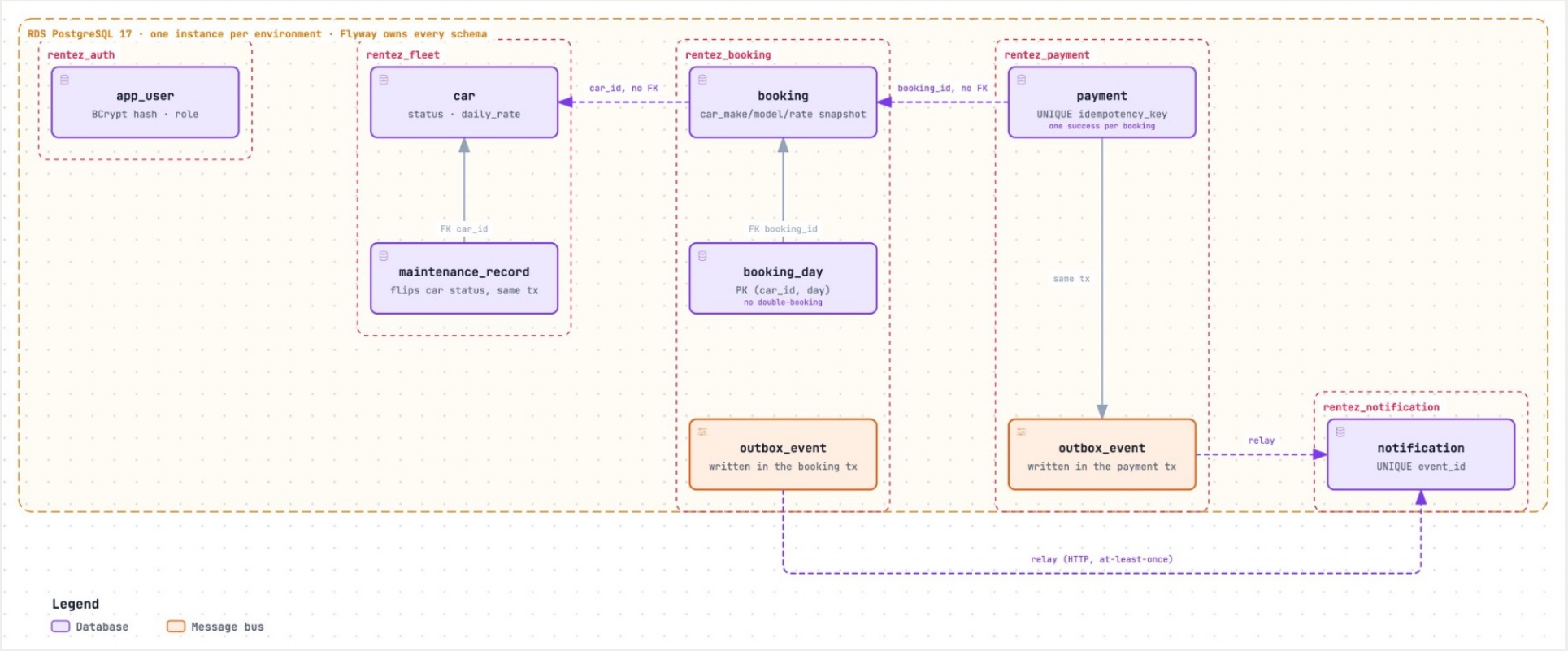
Persistence Design

Data store	Type	Owning service	Data held	Why
PostgreSQL 17 (RDS)	Relational	All 5	Own schema each	ACID, DB invariants
booking_day	PK (car_id, day)	reservation	Car per day	No double-booking
payment	UNIQUE keys	payment	Idempotency key	Safe retries
outbox_event	Table	reservation, payment	Pending events	Atomic with change

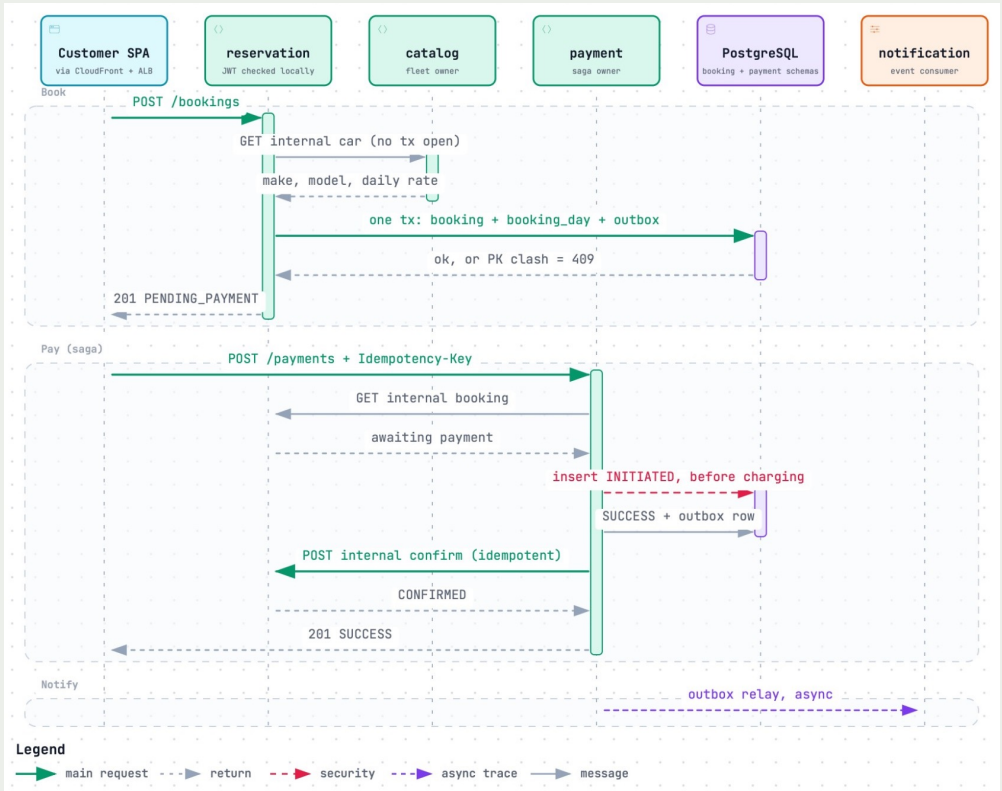
Consistency: schema per service; saga + outbox across services

Backup and retention: DB dumped to S3 on teardown; logs 7 days

Persistence: Schema per Service



Representative Use Case: Book and Pay



Flow

1. Book: one tx, or 409
2. Pay: INITIATED before charge
3. Idempotent confirm, else compensate
4. Outbox relays the event

Why representative: exercises almost every decision

Source Code and Artifact Management

</>

Repositories

- Monorepo on GitHub
- 5 standalone Maven services
- No shared library or parent POM

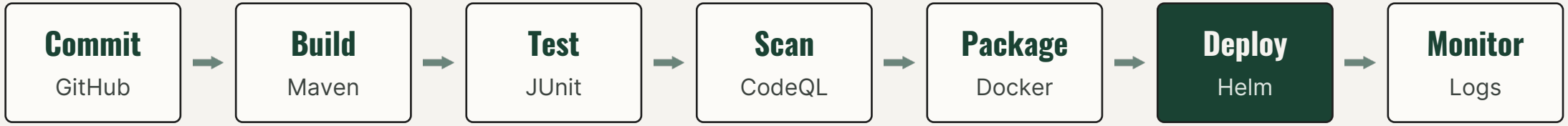
Branching and review

- feature/** → dev → main
- PR + green CI on dev and main
- sync-main auto-PRs; CODEOWNERS

Artifacts

- ECR: rentez-<service>
- Tagged short + full commit SHA
- Secrets in SSM; Helm values per service

DevOps Pipeline and Procedures



Environments and promotion

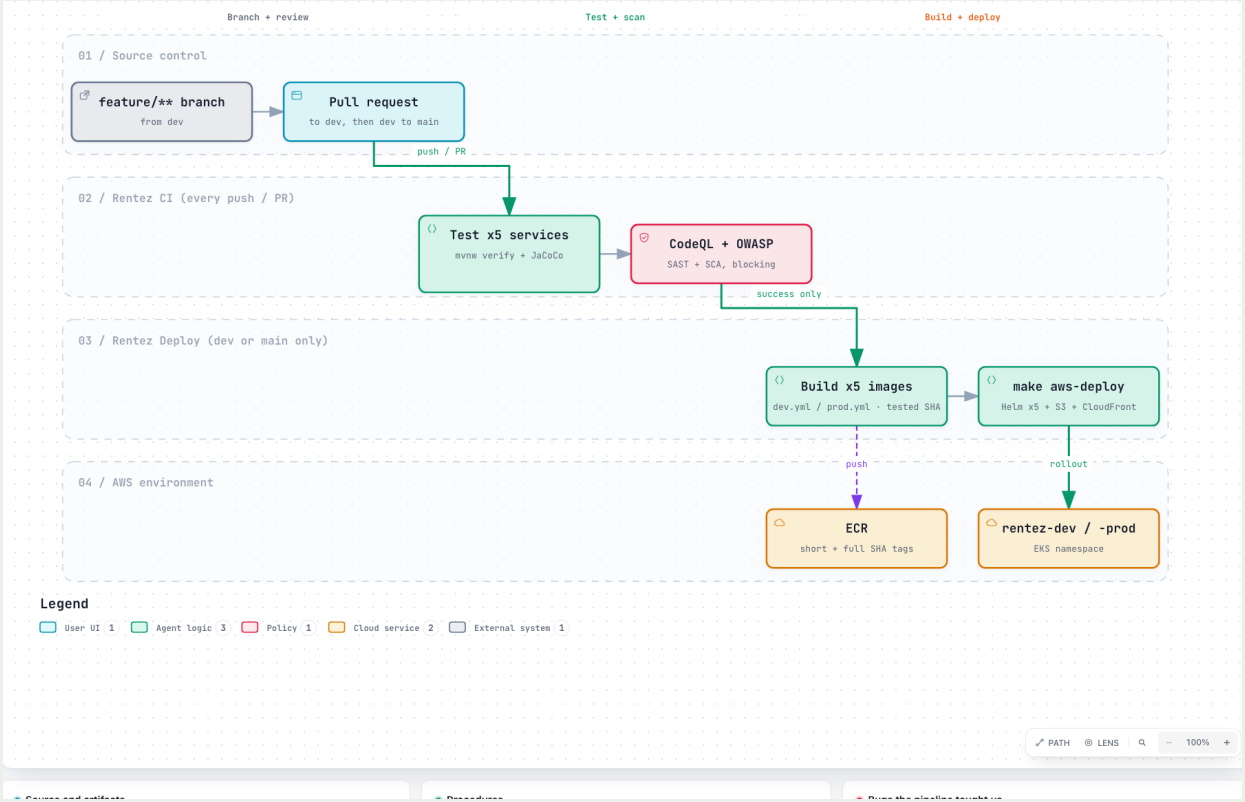
- feature/** → dev (dev env) → main (prod env)
- Deploy on CI success only, tested commit SHA
- eksctl + CloudFormation + Helm

Procedures

- Tests + CodeQL + OWASP block the merge
- Redeploy previous SHA with aws-deploy
- JWT and DB passwords in SSM

05 DEVOPS & DEVELOPMENT LIFECYCLE · 6 + 1 MIN

Pipeline: Branch to Running Pods



Technical Findings and Issues

Finding / issue	Root cause	Resolution	Lesson learnt
Deadlock at 50 actions/s, CPU < 1%	Audit used REQUIRES_NEW: 2nd connection per write	Audit joins the caller's tx	Green tests do not mean it scales
Pool starved under load	Connections held across HTTP calls	No tx open during remote calls	Never wait on the network holding a lock
Overload never drained	30 s pool wait, no HTTP timeouts	2 s pool timeout + HTTP timeouts	Fail fast so the system recovers
Every error returned 401	Error dispatch treated as anonymous	Permit ERROR dispatch	Masked errors hide the real cause
8 pods Pending for 15 min	Cluster Autoscaler v1.35 on k8s 1.31	Pinned v1.31.5 + version guard	Verify autoscaling actually fires
Dev deploys shipped main's code	workflow_run github.sha = default branch	Deploy head_sha of the tested run	Run the pipeline, don't trust it

NUS National University of Singapore | ISS

Before

<50



After

≥400

Actions/s supported locally after the five fixes: same hardware, same 3-connection pools, no new infrastructure

06 DEMO · 8 + 2 MIN

Performance on AWS

56

p95 ms at 100/s

AWS dev, through CloudFront

97

p99 ms at 100/s

Target < 1000 ms

0%

Error rate at 100/s

Target < 1%

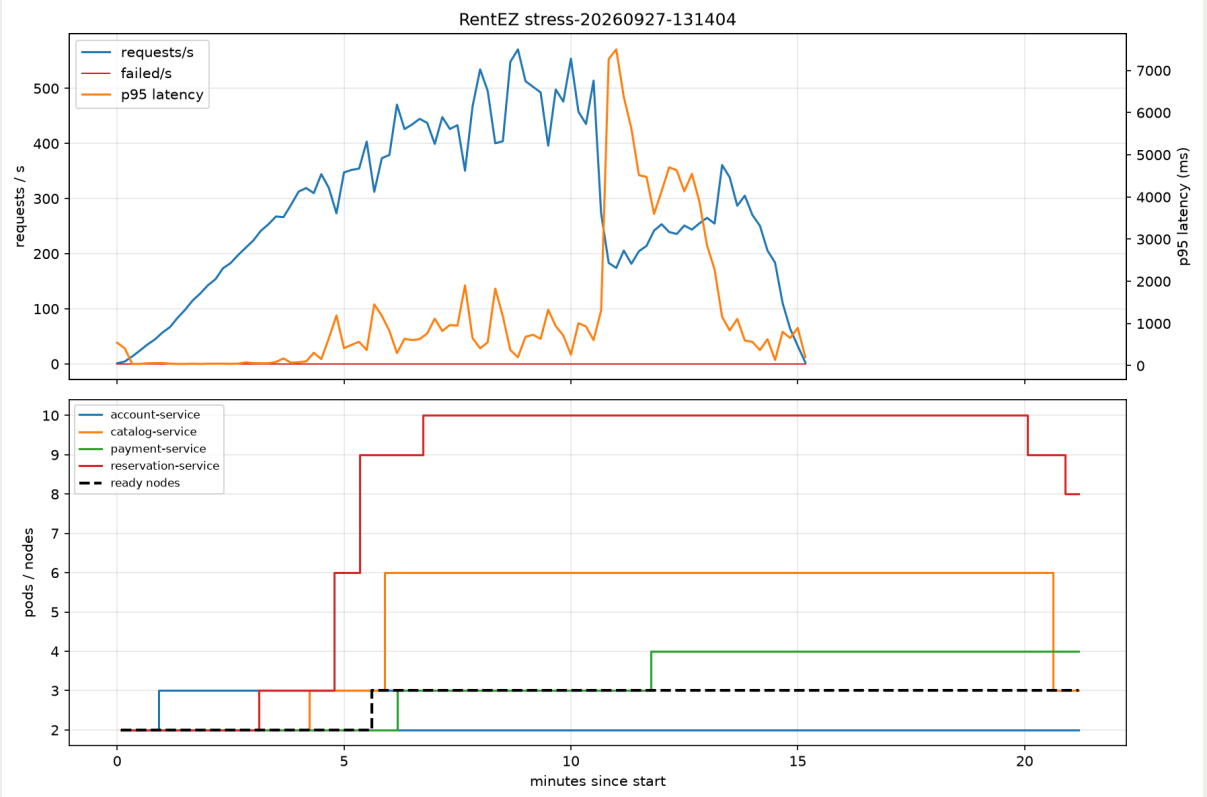
10

Reservation pods at peak (from 2)

Nodes 2 → 3 via Cluster Autoscaler

06 DEMO · 8 + 2 MIN

Scalability: Capacity Follows Demand



06 DEMO · 8 + 2 MIN

Demo Plan

▷ Key use cases

1. Book and pay · 2 min
2. Race: 201 + 409 · 1 min
3. Payment replay · 1 min

Quality attributes

Attribute	What we show	Tool
Scalability	HPA 2→10 pods, new node under a spike	k6 spike + kubectl watch
Performance	SLO thresholds pass at 100/s	k6 load test
Maintainability	Commit → CI → deploy, new SHA running	GitHub Actions
Resilience	Dashboard partial: true with payment down	kubectl scale

NUS National University of Singapore | ISS

Thank You

Questions and Answers

Group 2 · github.com/Uygnis/NUS-ISS-ASS-Practice-Module

